SESI ACOLHE

Modelagem do Banco de Dados

Etapa 2 • Modelo de dados, Diagrama Entidade-Relacionamento (DER) e Script SQL

Princípio do projeto
OUVIR → ACOLHER → ENCAMINHAR

Documento complementar à etapa de Ideação e Definição de Requisitos.

Tecnologia escolhida: SQLite

1. Objetivo

Este documento apresenta a proposta de modelagem do banco de dados da plataforma SESI Acolhe. O banco deverá organizar os usuários autorizados, os relatos enviados pelos estudantes, os registros de acompanhamento e as solicitações de atendimento, respeitando a privacidade e as regras de negócio definidas na etapa anterior.

A modelagem é uma proposta inicial e deve ser validada pelo grupo e pelo professor antes da implementação.

2. Modelo do banco de dados

O modelo utiliza quatro tabelas relacionadas. As chaves primárias (PK) identificam cada registro de forma única. As chaves estrangeiras (FK) conectam registros entre tabelas.

usuarios

Armazena profissionais e usuários que precisam de conta na plataforma.

Campo	Tipo	Regra / finalidade
id_usuario (PK)	INTEGER	Identificador único; autoincremento.
nome	TEXT	Nome do usuário.
email	TEXT	E-mail único para acesso.
senha_hash	TEXT	Hash seguro da senha; nunca guardar senha pura.
tipo	TEXT	Perfil, por exemplo: profissional ou administrador.

relatos

Armazena os relatos e seu estado de acompanhamento.

Campo	Tipo	Regra / finalidade
id_relato (PK)	INTEGER	Identificador interno.
codigo_acompanhamento	TEXT	Código único para consulta do andamento.
id_usuario (FK)	INTEGER	Usuário associado quando identificado; NULL para anônimo.
titulo	TEXT	Título breve do relato.
descricao	TEXT	Descrição da situação.
categoria	TEXT	Categoria, como bullying, assédio moral ou conflito.
anonimo	INTEGER	0 = identificado; 1 = anônimo.
data_criacao	TEXT	Data e hora de criação.
status	TEXT	Recebido, Em análise, Encaminhado, Em acompanhamento ou Finalizado.

atendimentos

Guarda o histórico de ações e atualizações relacionadas a um relato.

Campo	Tipo	Regra / finalidade
id_atendimento (PK)	INTEGER	Identificador do registro de acompanhamento.

Campo	Tipo	Regra / finalidade
id_relato (FK)	INTEGER	Relato ao qual o registro pertence.
id_responsavel (FK)	INTEGER	Profissional autorizado responsável pelo registro.
observacao	TEXT	Anotação necessária ao acompanhamento.
data_atendimento	TEXT	Data e hora do registro.
status	TEXT	Estado do caso após a atualização, quando aplicável.

solicitacoes_atendimento

Registra pedidos de atendimento feitos pelos estudantes.

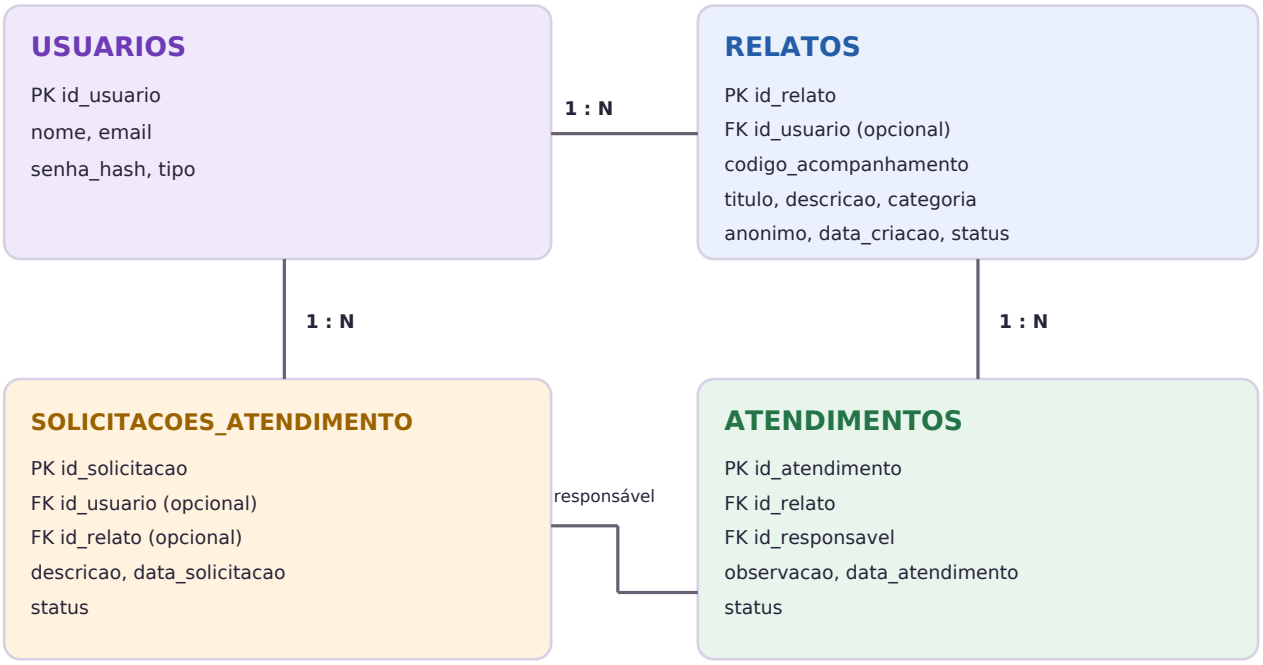
Campo	Tipo	Regra / finalidade
id_solicitacao (PK)	INTEGER	Identificador do pedido.
id_usuario (FK)	INTEGER	Usuário que solicitou, se identificado.
id_relato (FK)	INTEGER	Relato relacionado, se houver.
descricao	TEXT	Motivo ou informação necessária para o pedido.
data_solicitacao	TEXT	Data e hora do pedido.
status	TEXT	Pendente, Agendada, Atendida ou Cancelada.

3. Relacionamentos e cardinalidade

• Um usuário pode estar associado a vários relatos identificados. Relatos anônimos não precisam de usuário associado.
• Um relato pode possuir vários registros de atendimento ao longo do tempo.
• Um profissional pode registrar vários atendimentos.
• Um usuário pode criar várias solicitações de atendimento.
• Uma solicitação pode estar ligada a um relato, mas também pode existir sem relato associado.

4. DER — Diagrama Entidade-Relacionamento

O diagrama abaixo apresenta as entidades e suas ligações principais. PK indica chave primária e FK indica chave estrangeira.



Observação: o diagrama é conceitual e pode ser ajustado conforme as decisões finais do grupo. A associação de identidade a relatos anônimos deve ser impedida pela lógica da aplicação, não apenas pela interface.

5. Script SQL — SQLite

O script abaixo cria as quatro tabelas e define chaves, restrições e valores permitidos para os status. Para executar, copie o código para um arquivo chamado schema.sql e rode-o em um ambiente SQLite, como DB Browser for SQLite ou o terminal sqlite3.

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS usuarios (
    id_usuario INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    senha_hash TEXT NOT NULL,
    tipo TEXT NOT NULL
        CHECK (tipo IN ('profissional', 'administrador'))
);

CREATE TABLE IF NOT EXISTS relatos (
    id_relato INTEGER PRIMARY KEY AUTOINCREMENT,
    codigo_acompanhamento TEXT NOT NULL UNIQUE,
    id_usuario INTEGER NULL,
    titulo TEXT NOT NULL,
    descricao TEXT NOT NULL,
    categoria TEXT NOT NULL,
    anonimo INTEGER NOT NULL DEFAULT 1
        CHECK (anonimo IN (0, 1)),
    data_criacao TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status TEXT NOT NULL DEFAULT 'Recebido'
        CHECK (status IN (
            'Recebido', 'Em análise', 'Encaminhado',
            'Em acompanhamento', 'Finalizado'
        )),
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
        ON DELETE SET NULL,
    CHECK (
        (anonimo = 1 AND id_usuario IS NULL) OR
        (anonimo = 0 AND id_usuario IS NOT NULL)
    )
);

CREATE TABLE IF NOT EXISTS atendimentos (
    id_atendimento INTEGER PRIMARY KEY AUTOINCREMENT,
    id_relato INTEGER NOT NULL,
    id_responsavel INTEGER NOT NULL,
    observacao TEXT NOT NULL,
    data_atendimento TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status TEXT
        CHECK (status IS NULL OR status IN (
            'Recebido', 'Em análise', 'Encaminhado',
            'Em acompanhamento', 'Finalizado'
        )),
    FOREIGN KEY (id_relato) REFERENCES relatos(id_relato)
        ON DELETE CASCADE,
    FOREIGN KEY (id_responsavel) REFERENCES usuarios(id_usuario)
);

CREATE TABLE IF NOT EXISTS solicitacoes_atendimento (
    id_solicitacao INTEGER PRIMARY KEY AUTOINCREMENT,
    id_usuario INTEGER NULL,
    id_relato INTEGER NULL,
    descricao TEXT NOT NULL,
    data_solicitacao TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status TEXT NOT NULL DEFAULT 'Pendente'
        CHECK (status IN ('Pendente', 'Agendada', 'Atendida', 'Cancelada')),
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
        ON DELETE SET NULL,
    FOREIGN KEY (id_relato) REFERENCES relatos(id_relato)
        ON DELETE SET NULL
);

CREATE INDEX IF NOT EXISTS idx_relatos_status
    ON relatos(status);

CREATE INDEX IF NOT EXISTS idx_relatos_categoria
    ON relatos(categoria);

CREATE INDEX IF NOT EXISTS idx_atendimentos_relato
    ON atendimentos(id_relato);
```

6. Regras de segurança e integridade

1. As senhas devem ser armazenadas como hash seguro, gerado pela aplicação; nunca salvar a senha original.
2. O acesso ao conteúdo dos relatos deve ser limitado a profissionais autorizados.
3. Para um relato anônimo, o campo id_usuario deve ficar vazio e o sistema não deve associar a identidade do estudante ao relato.
4. O código de acompanhamento deve ser aleatório, único e difícil de adivinhar; não deve ser simplesmente o número sequencial do registro.
5. A aplicação deve validar permissões antes de mostrar, alterar ou encaminhar qualquer relato.
6. O banco não substitui os protocolos de proteção e encaminhamento definidos pela escola.

7. Conclusão

A modelagem proposta organiza os dados principais do SESI Acolhe e permite registrar relatos, acompanhar mudanças de status, manter histórico de atendimento e receber solicitações. O DER mostra os relacionamentos, enquanto o script SQL implementa a estrutura inicial no SQLite. Antes da entrega final, o grupo deve revisar os nomes dos campos, as regras de anonimato e os perfis de acesso com o professor.